

ORGNZ-03: Bucket Strategy and Design

Series: ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 3 of 10 | **Created:** January 2026 | **Last Updated:** 09/24/2026

Overview

A well-defined bucket strategy optimizes query performance, controls costs, and ensures compliance. This notebook covers naming conventions, retention planning, organizational alignment, and common bucket design patterns.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	<code>storage:bucket-definitions:read</code> , <code>storage:logs:read</code>
Knowledge	Completed ORGNZ-02 (Understanding Grail Buckets)
Data	At least 1 hour of log data

Table of Contents

- 1. [Bucket Naming Conventions](#)
- 2. [Retention Strategy](#)
- 3. [Bucket Design Patterns](#)
- 4. [Cost Control and Attribution](#)
- 5. [Analyzing Bucket Usage](#)

6. [Bucket Strategy Considerations](#)
7. [Routing Data to Buckets](#)
8. [Bucket Design Checklist](#)
9. [Query Billing Model](#)

Learning Objectives

By the end of this notebook, you will:

- Design effective bucket naming conventions
- Plan retention strategies for different use cases
- Understand cost implications of bucket design
- Apply organizational alignment patterns

Bucket Naming Conventions

Recommended Naming Patterns

Pattern	Format	Example	Use Case
Provider-based	<code><provider>_<type>_<retention></code>	<code>aws_logs_35d</code>	Multi-cloud environments
LOB-based	<code><provider>_<type>_<lob></code>	<code>aws_logs_finance</code>	Line of business cost attribution
Team-based	<code>team_<name>_<type></code>	<code>team_platform_logs</code>	Team ownership and billing
Compliance	<code><regulation>_<type>_<retention></code>	<code>hipaa_logs_7y</code>	Regulatory requirements

Detailed Examples

Bucket Name	Provider	Data Type	Retention	Org Unit
aws_logs_35d_platform	AWS	logs	35 days	Platform team
azure_spans_14d_finance	Azure	spans	14 days	Finance
gcp_spans_14d_checkout	GCP	spans	14 days	Checkout service
audit_logs_365d_compliance	Any	logs	365 days	Compliance
debug_logs_7d	Any	logs	7 days	Development

Naming Rules Reminder

Rule	Requirement
First character	Must be a letter
Allowed characters	Lowercase alphanumeric, underscores, hyphens
Case	Lowercase only
Reserved names	Cannot use default_* prefix
Immutability	Names cannot be changed after creation

Retention Strategy

Retention Period Guidelines

Use Case	Recommended Retention	Rationale
Debug/Development	3-7 days	Short-term troubleshooting, high volume
Standard Operations	35 days	Monthly trends, incident response
Performance Analysis	60-90 days	Quarterly comparisons
Compliance/Audit	365-3657 days	Regulatory requirements

Retention by Data Type

Data Type	Typical Retention	Notes
Debug logs	3-7 days	High volume, low long-term value
Application logs	35-90 days	Operational analysis
Audit logs	1-7 years	Compliance requirements
Metrics	Fixed — <code>default_metrics</code> , ~15 months	Not configurable per bucket: metrics have no custom buckets
Spans	10-35 days	APM troubleshooting
Security events	90-365 days	Security investigations

Bucket Design Patterns

Pattern 1: Long-Term Compliance Data

Store critical audit information for extended periods:

```
Bucket: compliance_audit_logs
Table: logs
Retention: 1825 days (5 years)
Use case: SOX compliance, security audits
Route via: OpenPipeline filter on audit-level logs
```

Pattern 2: Short-Term Debug Logs

Reduce costs by storing verbose logs briefly:

```
Bucket: debug_logs_7d
Table: logs
Retention: 7 days
Use case: Development troubleshooting
Route via: OpenPipeline filter on loglevel=DEBUG
```

Pattern 3: Team Isolation

Separate buckets for team-based access and cost attribution:

```
Bucket: team_platform_logs
Table: logs
Retention: 35 days
Use case: Platform team owns infrastructure logs
Access: IAM policy restricts to platform team
```

Pattern 4: Application-Specific Retention

Business units with specific retention needs:

```
Bucket: finance_app_logs
Table: logs
Retention: 180 days
Use case: Financial application compliance
Cost: Chargeback to Finance cost center
```

Cost Control and Attribution

Cost Factors

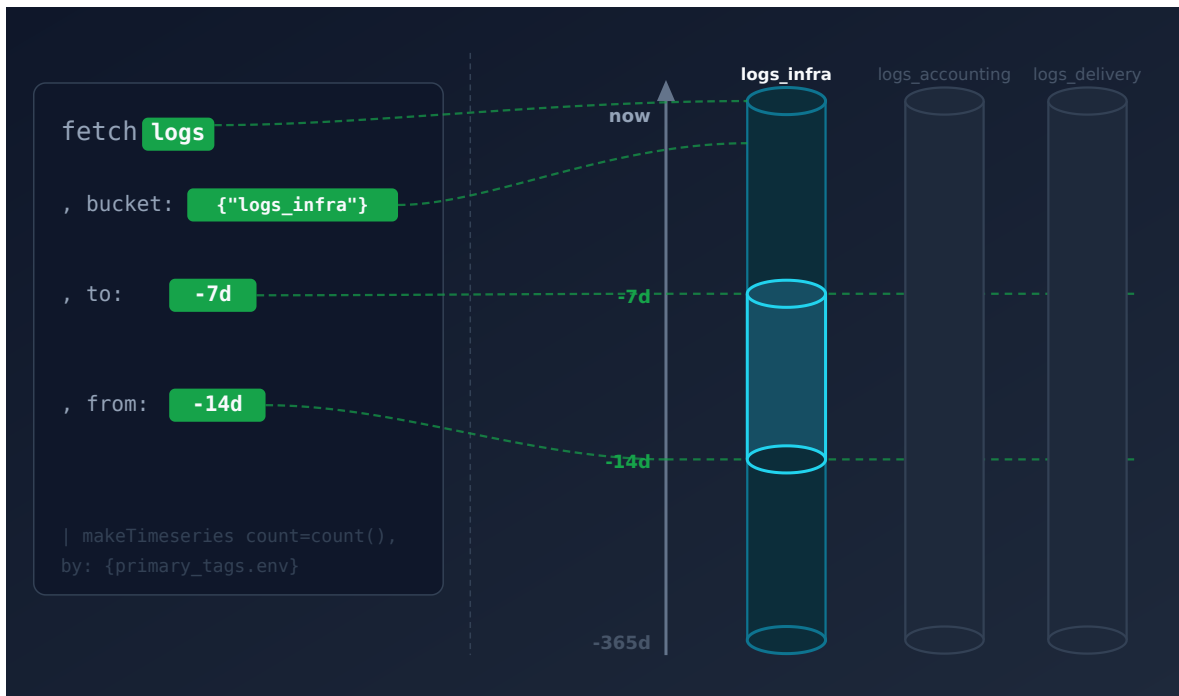
Factor	Impact on Cost
Ingest volume (GiB/day)	Direct relationship
Retention period	Storage costs increase with longer retention
Query frequency	Query costs accumulate

Cost Attribution Strategy

Organization Level	Bucket Strategy
Business Unit	One bucket per BU
Cost Center	Map buckets to cost centers
Cloud Account	Separate by AWS/Azure/GCP account

Organization Level	Bucket Strategy
Application	Critical apps get dedicated buckets

Analyzing Bucket Usage



DQL: Capacity Planning

Track ingest trends to validate routing rules and spot anomalies:

```
// Track log ingest trends by bucket over the last day – use for capacity
// planning and routing anomaly detection
// For multi-day capacity planning use the billing-usage events in FINOPS-01
// rather than a raw multi-day log scan
fetch logs, from:-24h
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h),
dt.system.bucket}
| sort time_bucket desc
```

Bucket Strategy Considerations

Access Control Options

Buckets can serve as an access control mechanism:

Access Method	Description	When to Use
Security context (record-level)	Fine-grained ABAC at the record level	Complex multi-team scenarios
Bucket-level IAM policies	Restrict access to entire buckets	Simple team isolation
Combined approach	Security context + bucket policies	Maximum control and flexibility

Bucket-Based Team Isolation Example:

```
Team A: Access to team_a_logs bucket only
Team B: Access to team_b_logs bucket only
Platform: Access to all buckets
```

Query Performance at Scale

Bucket sizing shapes queryability because `fetch` stops after 500 GB of uncompressed data by default (override with `scanLimitGBytes`):

Bucket Ingest	Impact
~1 TB/day	Optimal - can query ~12 hours of data
1-2 TB/day	Acceptable — reduced query window
>2 TB/day	Split — Dynatrace recommends splitting above this threshold

Data Immutability

Operation	Possible?	Alternative
Move data between buckets	No	Route new data via OpenPipeline
Selective record deletion	No	Wait for retention to expire
Bucket deletion	Yes	Deletes ALL data permanently

Routing Data to Buckets

Buckets are assigned in OpenPipeline, in the **Storage** stage of a pipeline. That stage holds **Bucket assignment** processors — *"Assign records to the best-fit bucket."* — and the **No storage assignment** processor, and it runs **first match only**: *"Depending on the stage type, a processor returns either the first matching record or all matching records."* Each processor pairs a DQL matcher with a target bucket; the matcher *"defines the target of a processor via a DQL statement and narrows down the available data to the specific set you want to process."*

Corrected 09/24/2026. Earlier versions of this notebook showed an OpenPipeline processors: - type: route YAML with rules, destination and default. OpenPipeline has no such processor or setting — configure Storage-stage bucket assignment as below.

Add one **Bucket assignment** processor per destination, most specific first:

Matching condition (DQL)	Bucket
loglevel == "DEBUG"	debug_logs_7d
matchesPhrase(log.source, "audit")	audit_logs_365d
matchesValue(dt.host_group.id, "finance-*")	finance_app_logs

Write matchers with the functions the OpenPipeline matcher documents — `==`, `matchesValue`, `matchesPhrase`, `isNull` / `isNotNull`, `iAny` — rather than `contains` or `starts-with`. `matchesValue` accepts a leading or trailing `*` and *"works with multi-value attributes (matching any value), and supports wildcards"*.

There is no `default:` route to set. Where no bucket assignment applies, the record is not lost: *"If you haven't specified bucket assignment in the pipeline configuration, Dynatrace will send all ingested data to the default buckets."* Check unmatched records in the pipeline preview before relying on that for a custom pipeline.

Routing on primary fields

OneAgent enriches telemetry at the source with standardized **primary fields** from the Semantic Dictionary (`dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`) plus customer-defined **primary tags**. They appear as top-level fields on metrics, spans, logs,

business events and Smartscape entities, so bucket assignment can key on them directly (Latest Dynatrace):

Matching condition (DQL)	Bucket
<code>dt.cost.costcenter == "cc-1234"</code>	<code>finance_logs</code>
<code>matchesValue(dt.security_context, "pci")</code>	<code>pci_audit_logs_365d</code>
<code>dt.cost.product == "checkout"</code>	<code>checkout_logs</code>

Use `matchesValue` rather than `==` on `dt.security_context` : the field can hold an array, and the equality operator *"doesn't operate on elements being part of multi-value attributes."* See **ORGNZ-06: Security Context** for how the value is set and **OPLOGS / OPIPE** for pipeline configuration.

Logs from official Dynatrace database extensions already have a built-in bucket, `default_database_monitoring` , which keeps them out of `default_logs` for faster queries and tighter IAM scoping. The `default_*` prefix is reserved — only Dynatrace-provided buckets use it.

Smartscape Ownership integration. Smartscape entities now carry ownership information that Dynatrace Workflows can read directly — useful when bucket-routing decisions depend on which team owns the producing host or service. See **WFLOW** for the routing-on-ownership pattern.

Sources: [Processing in OpenPipeline \(DT docs\)](#), [DQL matcher in OpenPipeline \(DT docs\)](#), [Use Grail buckets to partition data \(DT docs\)](#). Matchers executed as DQL filters on a live tenant 09/24/2026.

Bucket Design Checklist

Use this checklist when planning buckets:

- ☐ Defined naming convention aligned with organization
- ☐ Identified retention requirements by data type
- ☐ Mapped organizational units to buckets for cost attribution
- ☐ Calculated expected ingest volumes (keep <1 TB/day)
- ☐ Planned for compliance requirements
- ☐ Documented bucket purposes
- ☐ Planned OpenPipeline routing rules

- [] Considered access control strategy (bucket vs security context)

Query Billing Model

Custom **log** buckets offer two query billing models:

Model	How Billed	Best For
Usage-based	Charged per GiB scanned at query time	Buckets queried infrequently or unpredictably
Retain with Included Queries	Queries over data inside the bucket's Included Queries period (10–35 days) are not charged separately; data older than that is queried usage-based	Buckets queried frequently or heavily by dashboards/alerts

"Customers can split a log bucket's retention period into two parts" — an Included Queries period of 10–35 days, inside an overall retention of up to 10 years. The choice is not permanent: "Alternatively, if the Retain with Included Query option does not meet your use case and requirements, you can reconfigure a bucket at any time to use individually billed on-demand queries without losing data."

For compliance and audit buckets queried daily by security dashboards, **Retain with Included Queries** typically lowers total cost. For debug or ephemeral buckets queried only during incidents, **Usage-based** avoids paying for query capacity you rarely use.

Excluding Logs from Storage

When you extract metrics from logs via OpenPipeline (e.g., parsing error rates into a metric), you may not need to retain the source log records at all. Configure a **No Storage Assignment** in the OpenPipeline Storage stage to drop matching records after processing — this avoids retaining data purely for a signal you've already transformed.

Use with caution: once records are dropped they cannot be recovered. Only use No Storage Assignment when metrics extraction fully captures the required signal.

Sources: [Log Analytics \(DPS\)](#) ([DT docs](#)).

Next Steps

Continue with the ORGNZ series:

- **ORGNZ-04**: Permissions in Grail Overview

References

- [Grail Buckets](#)
- [Data Retention](#)
- [OpenPipeline Routing](#)
- [Log Analytics \(DPS\) \(DT docs\)](#)

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.